

Dumb Ways to Solve Subset Sum: Penyelesaian Eksak Single-Threaded pada $N = 100$, $T \approx 1T$, Densitas $\approx 2,51$, dan Memori di Bawah 50 MB RAM pada Laptop Berbiaya Rendah

M. Diki Fahriza
Peneliti Independen
mdikifahriza3@gmail.com

Abstract

Subset Sum Problem (SSP) merupakan salah satu masalah NP-lengkap fundamental dengan penerapan luas dalam kriptanalisis, pemrograman bilangan bulat, dan optimasi kombinatorial. Penyelesai (*solver*) eksak praktis umumnya menghadapi tiga hambatan komputasi utama: pendekatan simetris Meet-in-the-Middle (MITM) membutuhkan memori $\mathcal{O}(2^{N/2})$ yang memicu cache thrashing parah saat $N \gtrsim 50$; pemrograman dinamis (DP) klasik memerlukan memori $\mathcal{O}(T)$ yang runtuh pada target berskala triliun ($T \geq 10^{12}$); serta ekstraksi banyak saksi solusi dari pohon branch-and-bound menuntut penelusuran ulang yang mahal. Makalah ini menyajikan Dumb SSP Solver, sebuah arsitektur penyelesai eksak berbasis C++20 yang mengintegrasikan empat mekanisme rekayasa: (1) mesin memoization tail-table asimetris pada m elemen aktif terkecil ($m = N - 1$ untuk $N \leq 20$, dan $m = 20$ dibatasi memori untuk $N > 20$), yang membatasi penggunaan memori pada 16,00 MB agar sepenuhnya muat di dalam cache CPU Level-3 (L3) sekaligus memangkas m tingkat rekursi terakhir melalui satu kali pencarian biner per daun; (2) mesin DP bitset paralel-word 64-bit yang dibatasi untuk target efektif kecil ($T \leq 1,5 \times 10^7$) dan kueri saksi tunggal (FindOne/DecisionOnly); (3) Zero-Sum Swap Extractor berderajat hingga yang memanen saksi tambahan melalui substitusi elemen simetris dan asimetris hingga derajat $p \leq 4$ dalam kompleksitas waktu polinomial tetap $\mathcal{O}(k^4 \log k + (N - k)^4 \log k)$; serta (4) verifikator black-box $\mathcal{O}(k)$ terisolasi (L7) yang memvalidasi batas indeks, keunikan elemen, dan kesamaan jumlah eksak di luar mesin pencarian. Seluruh waktu eksekusi yang dilaporkan diperoleh dari tolok ukur uji mandiri single-threaded yang sepenuhnya dapat direproduksi pada rentang $N = 5$ hingga $N = 100$ dengan target berskala triliun pada perangkat keras laptop komoditas dan dapat diverifikasi langsung melalui dumbsspCli.exe.

Index Terms

Subset Sum Problem, Meet-in-the-Middle, Rekayasa Algoritma, Lokalitas Cache, Dynamic Programming Bitset, Ruang Pencarian Lokal, Perturbasi Jumlah-Nol, Kompleksitas Terparameterisasi, Reprodutibilitas.

I. Pendahuluan

MASALAH Subset Sum Problem (SSP) adalah salah satu masalah paling mendasar dalam ilmu komputer teoretis, riset operasi, dan optimasi kombinatorial. Secara formal, diberikan sebuah multiset terurut dari N bilangan bulat positif $A = \{a_1, a_2, \dots, a_N\} \subset \mathbb{Z}^+$ dan bilangan bulat target $T \in \mathbb{Z}^+$, masalah keputusan SSP menanyakan apakah terdapat himpunan bagian indeks $S \subseteq \{1, 2, \dots, N\}$ sedemikian rupa sehingga penjumlahan elemen-elemen terpilih tepat sama dengan T :

$$\sum_{i \in S} a_i = T. \quad (1)$$

Tiga varian formulasi dipertimbangkan dalam makalah ini dan didukung langsung oleh implementasi melalui nilai SolveMode:

- 1) Varian Keputusan (DecisionOnly): menentukan keberadaan setidaknya satu himpunan bagian yang memenuhi, $\Phi(A, T) \in \{\text{BENAR}, \text{SALAH}\}$.

- 2) Varian Pencarian (FindOne/FindAll): merekonstruksi saksi biner eksplisit $x = (x_1, \dots, x_N) \in \{0, 1\}^N$ dengan $\sum_{i=1}^N a_i x_i = T$, baik saksi tunggal maupun sebanyak batas yang dikonfigurasi.
- 3) Varian Pencacahan (CountAll): menghitung kardinalitas

$$\mathcal{N}(A, T) = \left| \left\{ S \subseteq \{1, 2, \dots, N\} : \sum_{i \in S} a_i = T \right\} \right| \quad (2)$$

terhadap elemen aktif multiset A (Bagian II); lihat Catatan 2 untuk penanganan elemen bernilai nol.

A. Lanskap Teori Kompleksitas

Sejak karya monumental Karp pada tahun 1972 mengenai reduksibilitas kombinatorial [1], SSP memegang peran sentral dalam teori kompleksitas komputasi. Sebagai salah satu dari 21 masalah NP-lengkap Karp, SSP bersifat sulit (*intractable*) pada kasus terburuk di bawah asumsi $P \neq NP$, dan di bawah Strong Exponential Time Hypothesis (SETH), algoritma eksak untuk instans sembarang diperkirakan membutuhkan waktu $\mathcal{O}^*(2^{(1-\varepsilon)N})$.

Meskipun demikian, SSP tergolong NP-lengkap lemah (*weakly NP-complete*): ketika T dan nilai a_i dibatasi secara polinomial dalam N , pemrograman dinamis Bellman menyelesaikan SSP dalam waktu pseudo-polinomial $\mathcal{O}(N \cdot T)$ [2]. Algoritma pseudo-polinomial mendekati linier $\tilde{\mathcal{O}}(N + T)$ kemudian dikembangkan melalui pewarnaan acak (*color coding*), fungsi pembangkit, dan konvolusi FFT (Bringmann [3], Jin dan Wu [4], Chen dkk. [5]).

B. Hambatan Praktis pada Penyelesai yang Ada

- 1) Dinding Memori MITM: Horowitz–Sahni [6] membagi A menjadi dua bagian berukuran $\lfloor N/2 \rfloor$ dan $\lceil N/2 \rceil$, menghasilkan, mengurutkan, dan mencocokkan $2^{N/2}$ jumlah parsial dalam waktu dan ruang $\mathcal{O}(2^{N/2})$. Untuk $N \geq 50$, $2^{25} \approx 3,36 \times 10^7$ entri melampaui ukuran cache L3 CPU umumnya (16–64 MB pada komputer komoditas), memicu cache thrashing dan tekanan TLB. Schroepel dan Shamir [7] mereduksi ruang menjadi $\mathcal{O}(2^{N/4})$ dengan kompensasi *overhead* antrean prioritas.
- 2) Dinding Besaran Target: DP Bellman sangat efisien saat T berada pada kisaran jutaan, namun tabel DP berukuran $\mathcal{O}(T)$ mustahil dialokasikan saat T mencapai 10^{12} .
- 3) Penalti Ekstraksi Multi-Solusi: memperoleh $K \gg 1$ saksi berbeda melalui penelusuran ulang pohon branch-and-bound secara naif memakan biaya $\mathcal{O}(K \cdot 2^N)$ pada kasus terburuk.

C. Kontribusi dan Ruang Lingkup

Makalah ini mendokumentasikan arsitektur, implementasi, dan karakteristik empiris dari Dumb SSP Solver, yang dibangun menggunakan C++20 murni (dumbsspCore.hpp, antarmuka konsol sinkron dumbsspCli.cpp, dan antarmuka grafis Win32 dumbsspGui.cpp):

- Jalur pra-pemeriksaan deterministik multi-tahap (L0–L2) yang melakukan penyaringan domain, saringan hambatan FPB/GCD, dan uji kelayakan jendela kardinalitas dalam waktu polinomial sebelum pencarian eksponensial.
- Tail-table asimetris terbatas memori (L4) pada m elemen aktif terkecil, mempertahankan ukuran kompak 16,00 MB yang menetap di dalam cache L3 CPU untuk mempercepat evaluasi simpul daun.
- Mesin DP bitset paralel-word (L3), dengan ambang batas target kecil ($T \leq 1,5 \times 10^7$) untuk resolusi saksi tunggal berkecepatan tinggi.
- Zero-Sum Swap Extractor berderajat hingga (L8), menyediakan kerangka kerja polinomial tetap untuk memanen multi-solusi hingga derajat $p \leq 4$.
- Rangkaian tolok ukur yang dapat direproduksi: menyediakan dataset per instans yang mandiri serta perintah CLI eksplisit dari $N = 5$ hingga $N = 100$ dengan target triliunan, yang diarsipkan dalam benchmark.txt.

D. Struktur Makalah

Bagian II menyajikan model formal, teorema reduksi invariansi komplemen, dan metrik densitas informasi. Bagian III merinci jalur pra-pemeriksaan L0–L2 dan logika *router* adaptif (Algoritma 1). Bagian IV membahas DP bitset, mesin *tail-table*, dan *oracle* pemangkas. Bagian V memaparkan teori *Zero-Sum Swap* dan kompleksitasnya. Bagian VI menyajikan evaluasi eksperimental dan instruksi reproduksi. Bagian VII membahas antarmuka pengguna dan verifikator L7. Bagian VIII menyimpulkan makalah dan menguraikan arah penelitian mendatang.

II. Preliminari & Taksonomi Teori Kompleksitas

A. Formulasi Matematis Formal

Misalkan A_{raw} adalah multiset masukan berukuran N yang diberikan oleh pengguna. Penyelesai pertama-tama membangun multiset aktif:

$$A = \{a \in A_{\text{raw}} : a \neq 0 \wedge a \leq T\}, \quad (3)$$

dengan membuang elemen bernilai nol (karena tidak mengubah jumlah) dan elemen yang lebih besar dari T (karena elemen positif yang melebihi T tidak dapat menjadi bagian dari solusi eksak). Elemen-elemen A kemudian diurutkan menurun: $a_1 \geq a_2 \geq \dots \geq a_n > 0$ dengan $n = |A| \leq N$. Jumlah agregat multiset aktif dinyatakan sebagai $\sigma(A) = \sum_{i=1}^n a_i$.

Lemma 1 (Keabsahan Penyaringan Domain). Untuk setiap $S_{\text{raw}} \subseteq A_{\text{raw}}$ dengan $\sum S_{\text{raw}} = T$: (i) elemen nol dapat ditambahkan atau dihilangkan dari S_{raw} tanpa mengubah hasil jumlah; (ii) S_{raw} tidak memuat elemen $a > T$ karena semua elemen bernilai positif.

Proof. (ii) berlaku karena semua elemen bilangan bulat non-negatif; jika $a \in S_{\text{raw}}$ dan $a > T$, maka $\sum S_{\text{raw}} \geq a > T$, bertentangan dengan $\sum S_{\text{raw}} = T$. Poin (i) terbukti langsung dari sifat identitas penjumlahan 0. $\square$

Kandidat solusi direpresentasikan sebagai vektor biner $x = (x_1, \dots, x_n) \in \{0, 1\}^n$ terhadap multiset aktif, dengan fungsi bobot $f(x) = \sum_{i=1}^n a_i x_i$.

B. Teorema Reduksi Invariansi Komplemen

Teorema 1 (Reduksi Invariansi Komplemen). Untuk instans aktif (A, T) dengan $T > \frac{1}{2}\sigma(A)$ dan $T \leq \sigma(A)$, pencarian $x \in \{0, 1\}^n$ dengan $\sum_{i=1}^n a_i x_i = T$ ekuivalen dengan mencari komplemen $x' = \mathbf{1} - x$ sedemikian sehingga:

$$\sum_{i=1}^n a_i x'_i = T' = \sigma(A) - T, \quad T' < \frac{1}{2}\sigma(A). \quad (4)$$

Diberikan saksi tersertifikasi untuk T' , solusi x dipulihkan dalam waktu $\mathcal{O}(n)$ melalui $x_i = 1 - x'_i$.

Proof. Misalkan $x'_i = 1 - x_i$. Maka $\sum_i a_i x'_i = \sum_i a_i - \sum_i a_i x_i = \sigma(A) - T = T'$. Karena $T > \frac{1}{2}\sigma(A)$, diperoleh $T' = \sigma(A) - T < \frac{1}{2}\sigma(A)$. Arah sebaliknya bersifat simetris, menghasilkan pemetaan bijektif antar kedua himpunan solusi. $\square$

Transformasi ini diterapkan pada fungsi `Instance::normalize()`: nilai `effective_target` diatur ke $\sigma(A) - T$ setiap kali $T > \lfloor \sigma(A)/2 \rfloor$ dan $\sigma(A) \geq T$.

C. Densitas Informasi Telemetris

Implementasi menghitung skor densitas informasi:

$$d = \frac{n}{\log_2(a_1 + 1)}, \quad n = |A|, \quad a_1 = \max A, \quad (5)$$

serta bendera `strong_structure` dari jendela kardinalitas dan FPB.

Catatan 1 (Peran Diagnostik Densitas Informasi). Densitas informasi d dan bendera struktur dievaluasi sebagai metrik informatif pada panel telemetri untuk mengkarakterisasi tingkat kesulitan teoritis instans

TABLE I
Klasifikasi Teori Kompleksitas Arsitektur Penyelesai

Lapisan / Modul	Kelas	Waktu	Ruang	Mekanisme
L0: Filter & Normalizer	P	$\mathcal{O}(N)$	$\mathcal{O}(n)$	Eliminasi nol/elemen berlebih, reduksi komplemen
L1: Saringan FPB/GCD	P	$\mathcal{O}(n \log(\min A_i))$	$\mathcal{O}(1)$	$T \not\equiv 0 \pmod{\gcd(A)} \Rightarrow \text{UNSAT}$
L2: Profiler Kardinalitas	P	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Pengurutan, jumlah sufiks, jendela $[k_{\min}, k_{\max}]$
L3: DP Bitset Paralel-Word	Pseudo-P.	$\mathcal{O}(nT/64 + T)$	$\mathcal{O}(T)$	Pergeseran bit 64-bit; aktif jika $T \leq 1,5 \times 10^7$, mode saksi tunggal
L4: Tail-Table Asimetris	Exp. Param.	$\mathcal{O}(m2^m + 2^{n-m})$	$\mathcal{O}(2^m)$	Tabel pra-komputasi pada m elemen terkecil (Bagian IV)
L5a: Pemangkas Batas Sufiks	P	$\mathcal{O}(1)/\text{simpul}$	$\mathcal{O}(n)$	$\text{rem} > \text{suffix}[i] \Rightarrow \text{pangkas cabang}$
L5b: Oracle Kardinalitas Lokal	P	$\mathcal{O}(n-i)/\text{simpul}$	$\mathcal{O}(n)$	Pencarian biner + evaluasi ulang $[k_{\min}, k_{\max}]$ lokal
L6: Pembatalan Kooperatif	P	$\mathcal{O}(1)$ amort.	$\mathcal{O}(1)$	Flag atomik, diperiksa tiap 4096 simpul evaluasi
L7: Verifikator Independen	P	$\mathcal{O}(k)$	$\mathcal{O}(k)$	Validasi terisolasi: batas indeks, keunikan, $\sum a_j \stackrel{?}{=} T$
L8: Ekstraktor Swap Jumlah-Nol	P (derajat tetap)	$\mathcal{O}(k^4 \log k + (n-k)^4 \log k)$	$\mathcal{O}(k^4 + (n-k)^4)$	Pertukaran $p \leq 4$ elemen per sisi (Bagian V)

sesuai literatur transisi fase klasik [8], [9]. Pemilihan strategi penyelesaian diatur secara deterministik oleh batas memori dan target pada Algoritma 1.

Sebagai konteks, rezim klasik terbagi menjadi: densitas rendah ($d < 0,9408$), di mana instans acak terbukti dapat direduksi dalam waktu polinomial ke pencarian vektor pendek pada kisi $(N+1)$ -dimensi via algoritma LLL [8], [9]; densitas kritis ($d \approx 1$), di mana pencarian eksak secara asimtotik paling sulit; dan densitas tinggi ($d > 1,2$), di mana prinsip sarang merpati menghasilkan banyak solusi redundan sehingga pemrograman dinamis sangat efektif.

D. Taksonomi Teori Kompleksitas Arsitektur Penyelesai

Tabel I mengklasifikasikan setiap modul penyelesai berdasarkan kompleksitas waktu dan ruang yang diimplementasikan.

III. Jalur Pra-Pemeriksaan dan Router Adaptif

Penyelesai mengeksekusi tiga lapisan pra-pemeriksaan deterministik terlebih dahulu sebelum *router* menentukan mesin pencarian utama.

A. Lapisan Pra-Pemrosesan, Saringan, dan Profiling (L0–L2)

- 1) L0 (Filter & Normalizer): membangun A dari A_{raw} berdasarkan Persamaan (3), menghitung $\sigma(A)$, dan menerapkan Teorema 1 saat $T > \frac{1}{2}\sigma(A)$.
- 2) L1 (Saringan FPB/GCD): menghitung $g = \gcd(a_1, \dots, a_n)$ dalam waktu $\mathcal{O}(n \log(\min A_i))$.
Lemma 2 (Hambatan Modular). Jika $T \not\equiv 0 \pmod{g}$, maka tidak ada himpunan bagian $S \subseteq A$ yang berjumlah T .

Proof. $\sum_{i \in S} a_i = g \sum_{i \in S} (a_i/g)$ merupakan kelipatan dari g untuk sembarang S . $\square$

- 3) L2 (Profiler Kardinalitas): mengurutkan A secara menurun, menghitung jumlah sufiks $\text{suffix}[i] = \sum_{j=i}^n a_j$, serta menentukan batas jendela kardinalitas:

$$k_{\min} = \min \left\{ k : \sum_{i=1}^k a_i \geq T_{\text{eff}} \right\}, \quad (6)$$

$$k_{\max} = \max \left\{ k : \sum_{i=n-k+1}^n a_i \leq T_{\text{eff}} \right\}, \quad (7)$$

di mana $k_{\min}$ adalah jumlah elemen terbesar paling sedikit untuk mencapai T_{eff} , dan $k_{\max}$ adalah jumlah elemen terkecil terbanyak tanpa melampaui T_{eff} . Jika $k_{\min} > k_{\max}$, instans tersertifikasi UNSAT.

B. Logika Keputusan Router Adaptif

Algoritma 1 menyajikan logika pemilihan strategi penyelesaian berdasarkan ambang batas memori dan target efektif.

Algorithm 1 Logika Pemilihan Strategi Router Adaptif

Require: Multiset aktif A , target efektif T_{eff} , anggaran memori M_{limit} (MB), mode $\mathcal{M}$

Ensure: Strategi $\mathcal{S} \in \{\text{TrivialPreCheck}, \text{BitsetDP}, \text{HybridTailTable}\}$

```

1: if  $T_{\text{eff}} = 0$  then
2:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (solusi himpunan kosong)
3: end if
4: if  $A = \emptyset$  or  $T_{\text{eff}} > \sigma(A)$  then
5:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: melebihi total jumlah)
6: end if
7: if  $g > 1$  and  $T_{\text{eff}} \bmod g \neq 0$  then
8:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: saringan FPB)
9: end if
10: if semua  $a_i$  genap and  $T_{\text{eff}}$  ganjil then
11:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: paritas berlawanan)
12: end if
13: if  $k_{\min} > k_{\max}$  then
14:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: jendela kardinalitas kosong)
15: end if
16:  $\text{Mem}_{\text{DP}} \leftarrow \lceil (T_{\text{eff}} + 1)/64 \rceil \times 8 + (T_{\text{eff}} + 1) \times 4$ 
17: if  $T_{\text{eff}} \leq 1,5 \times 10^7$  and  $\text{Mem}_{\text{DP}} < \frac{1}{2} M_{\text{limit}} \cdot 2^{20}$  and  $\mathcal{M} \in \{\text{FindOne}, \text{DecisionOnly}\}$  then
18:   return  $\mathcal{S} \leftarrow \text{BitsetDP}$  (Lapisan L3)
19: else
20:   return  $\mathcal{S} \leftarrow \text{HybridTailTable}$  (Lapisan L4–L5)
21: end if

```

C. Mode Operasional yang Didukung

- FindOne / DecisionOnly: pencarian langsung kembali begitu kecocokan daun pertama ditemukan melalui flag `solution_found`.
- FindAll: setelah saksi pertama ditemukan, jalur utama menyerahkan proses ke ekstraktor L8 untuk memanen saksi tambahan hingga batas `max_solutions` (default 5000).
- CountAll: menghitung total solusi melalui akumulator 128-bit (`u128 solution_count`) tanpa mengalokasikan vektor saksi pada setiap solusi tambahan.

Catatan 2 (Skalabilitas Kombinatorial untuk Elemen Bernilai Nol). Elemen bernilai nol ($a_i = 0$) difilter pada tahap L0 untuk menjaga efisiensi pohon pencarian. Pada mode pencacahan, jumlah solusi total pada multiset awal secara aljabar memenuhi $\mathcal{N}(A_{\text{raw}}, T) = 2^Z \cdot \mathcal{N}(A, T)$, dengan Z menyatakan banyaknya elemen bernilai nol.

IV. Rekayasa Algoritma Berkesadaran Perangkat Keras

A. DP Bitset Paralel-Word 64-Bit (L3)

Saat diarahkan ke L3, ketercapaian target disimpan sebagai larik kata $\text{DP}[0..W-1]$ berukuran $W = \lceil (T_{\text{eff}} + 1)/64 \rceil$ kata 64-bit. Untuk setiap elemen v , transisi status dilakukan via pergeseran bit multi-word:

$$w_{\text{shift}} = \lfloor v/64 \rfloor, \quad b_{\text{rem}} = v \bmod 64, \quad (8)$$

$$\text{DP}'[w] = \text{DP}[w] \vee (\text{DP}[w - w_{\text{shift}}] \ll b_{\text{rem}}) \vee (\text{DP}[w - w_{\text{shift}} - 1] \gg (64 - b_{\text{rem}})). \quad (9)$$

Larik `parent[t]` 32-bit menyimpan jejak indeks elemen untuk merekonstruksi solusi dalam $\mathcal{O}(k)$ langkah.

B. Memoization Tail-Table Asimetris dalam Cache L3 CPU (L4)

Pendekatan MITM simetris [6] membagi n elemen menjadi 50:50. Lapisan L4 sebaliknya melakukan pra-komputasi hanya pada m elemen aktif terkecil, di mana nilai m ditentukan oleh:

$$m(n, M_{\text{limit}}) = \begin{cases} 0, & n \leq 1, \\ n - 1, & 2 \leq n \leq 20, \\ \max \left(12, \max \{ m' \leq 20 : 16 \cdot 2^{m'} \leq \frac{M_{\text{limit}} \cdot 2^{20}}{4} \} \right), & n > 20. \end{cases} \quad (10)$$

Pada batas memori default $M_{\text{limit}} = 4096$ MB, nilai $m = 20$ untuk semua $n > 20$. Tabel Tail = $\{a_{n-m+1}, \dots, a_n\}$ memuat tepat 2^m entri pasangan (u64 sum, u32 mask) dengan struktur 16-byte:

$$\text{Memori}(\text{Tabel}_{m=20}) = 2^{20} \times 16 \text{ byte} = 16,777,216 \text{ byte} = 16,00 \text{ MB}, \quad (11)$$

yang muat seutuhnya di dalam cache L3 CPU modern (16–64 MB). Tabel diisi dengan mengevaluasi kombinasi bit 2^m dalam waktu $\mathcal{O}(m 2^m)$ dan diurutkan dalam $\mathcal{O}(2^m \log 2^m)$ untuk pencarian biner cepat.

C. Pemangkasan Sufiks dan Kardinalitas pada DFS (L5)

Dua pemeriksaan pemangkasan berjalan di setiap simpul DFS (i, rem):

- L5a (Batas Sufiks, $\mathcal{O}(1)$): jika $\text{rem} > \text{suffix}[i] = \sum_{j=i}^n a_j$, seluruh sub-pohon langsung dipangkas.
- L5b (Oracle Kardinalitas Lokal): melakukan pencarian biner elemen terbesar yang valid, memeriksa ketersediaan elemen minimal, dan membangun jendela kardinalitas lokal $[k_{\min}^{(i)}, k_{\max}^{(i)}]$ dalam waktu $\mathcal{O}(n - i) = \mathcal{O}(N)$.

D. Pembatalan Kooperatif dan Titik Batas Waktu (L6)

Kontrol eksekusi disediakan oleh Lapisan L6 melalui flag atomik tunggal (`std::atomic<bool> stop_flag`). Pencarian DFS memeriksa flag ini setiap 4096 simpul terhadap batas waktu (`time_limit_ms`). GUI (`dumbsspGui.cpp`) menjalankan pencarian pada satu `std::thread` terdedikasi agar antarmuka tidak membeku, sedangkan CLI (`dumbsspCli.cpp`) berjalan murni sinkron dan single-threaded untuk memastikan determinisme dan bebas *jitter*.

V. Ekstraktor Swap Jumlah-Nol Berderajat Hingga

A. Fondasi Teoretis

Misalkan $S_0 \subseteq \{1, \dots, n\}$ adalah saksi awal dengan $\sum_{i \in S_0} a_i = T_{\text{eff}}$, $U \subset S_0$ adalah himpunan bagian yang dikeluarkan, dan $V \subset (\{1, \dots, n\} \setminus S_0)$ adalah himpunan bagian yang dimasukkan, menghasilkan $S' = (S_0 \setminus U) \cup V$.

Teorema 2 (Invariansi Delta Jumlah-Nol). S' adalah solusi eksak yang valid jika dan hanya jika $\Delta(U, V) = \sum_{v \in V} a_v - \sum_{u \in U} a_u = 0$.

Proof. $\sum_{j \in S'} a_j = (\sum_{j \in S_0} a_j - \sum_{u \in U} a_u) + \sum_{v \in V} a_v = T_{\text{eff}} + \Delta(U, V)$, yang tepat sama dengan T_{eff} jika dan hanya jika $\Delta(U, V) = 0$. $\square$

Teorema 3 (Jarak Hamming Genap). Jika $|U| = |V| = p$, jarak Hamming antara vektor indikator S_0 dan S' adalah $d_H = |U| + |V| = 2p$.

B. Ekstraksi Perturbasi Berderajat Hingga ($p \leq 4$)

Ekstraktor Swap Jumlah-Nol (`ZeroSumSwapExtractor::extract`) mengeksplorasi substitusi elemen hingga derajat $p \leq 4$ pada himpunan internal $S_{\text{in}} = S_0$ dan himpunan eksternal $S_{\text{out}} = A \setminus S_0$:

- 1) Menghitung semua $U \subset S_0$ dengan $1 \leq |U| \leq 4$: menghasilkan $\Theta(k^4)$ kombinasi, diurutkan dalam waktu $\Theta(k^4 \log k)$.

- 2) Menghitung semua $V \subset S_{\text{out}}$ dengan $1 \leq |V| \leq 4$ ($\Theta((n-k)^4)$ kombinasi); untuk setiap V , dilakukan pencarian biner pada larik U terurut untuk menemukan $\sum V = \sum U$ dalam waktu $\mathcal{O}(\log k)$ per pengujian.

Total kompleksitas ekstraksi adalah

$$\mathcal{O}(k^4 \log k + (n-k)^4 \log k), \quad (12)$$

yang merupakan prosedur polinomial berderajat tetap dalam kelas P.

VI. Evaluasi Eksperimental yang Dapat Direproduksi

Evaluasi eksperimental didasarkan pada transkrip uji mandiri 20 instans yang diarsipkan pada benchmark.txt, mencakup ukuran $N = 5, 10, 15, \dots, 100$ dengan target berskala triliun pada mode FindOne dan diverifikasi ulang secara independen oleh Lapisan L7. Setiap baris pada Tabel II dapat direproduksi langsung menggunakan dumbsspCli.exe.

A. Platform Pengujian Perangkat Keras

Seluruh tolok ukur diukur pada komputer laptop berbiaya rendah dengan konfigurasi sistem sebagai berikut:

- Model Sistem: Acer Aspire A314-35 (BIOS v1.09 UEFI).
- Prosesor (CPU): Intel(R) Celeron(R) N5100 @ 1.10 GHz (4 core, 4 thread, 4 MB L3 cache, TDP 6 W).
- Memori Sistem (RAM): 8192 MB (8,0 GB) DDR4 RAM.
- Sistem Operasi: Microsoft Windows 11 Pro 64-bit (Build 26200).
- Model Kompilasi & Eksekusi: MinGW-w64 GCC (-O3 -std=c++17), berjalan secara murni single-threaded sinkron via dumbsspCli.exe.

B. Instruksi Reproduksi

Reproduksi seluruh klaim kuantitatif dapat dilakukan via antarmuka baris perintah (dumbsspCli.exe):

dumbsspCli.exe "a_1, a_2, ..., a_N" T [findone|findall|count|decision]

Contoh untuk baris $N = 20$:

dumbsspCli.exe "879273641953, 844911863763, 148075117490, ..., 998161222768" 3972117833710 findone

C. Diskusi Hasil

Seluruh 20 instans berhasil diselesaikan dan lolos validasi independen L7. Terlihat bahwa seluruh instans menggunakan jalur L4 karena target berskala triliun melampaui batas $1,5 \times 10^7$ pada L3. Variasi waktu eksekusi dipengaruhi oleh efektivitas pemangkasan L5a/L5b pada struktur angka acak tiap instans.

D. Perbandingan Kualitatif dengan Metode Klasik

Tabel III menyajikan perbandingan paradigma penyelesaian subset sum berdasarkan literatur komputasi.

VII. Antarmuka Pengguna & Verifikasi Independen

A. Arsitektur Antarmuka Pengguna

Penyelesai menyediakan dua antarmuka: antarmuka grafis Win32 (dumbsspGui.cpp) untuk visualisasi langsung dan pemantauan telemetry; serta antarmuka konsol (dumbsspCli.exe) untuk pengujian tolok ukur saintifik presisi tinggi bebas beban tampilan jendela.

B. Verifikator Black-Box Independen L7

Lapisan L7 (verify_witness_independently) berjalan di luar mesin pencarian. Menggunakan akumulasi 128-bit, L7 memverifikasi: (i) indeks berada dalam rentang $[0, N]$; (ii) tidak ada indeks yang berulang; (iii) nilai elemen cocok dengan A_{raw} ; dan (iv) jumlah total tepat sama dengan T .

TABLE II
Hasil Uji Mandiri: FindOne pada Instans Acak Target Triliunan (Sumber: benchmark.txt)

N	Ukuran Saksi k	Waktu Eksekusi	Verifikator L7
5	2	0,028 ms	OK
10	3	0,053 ms	OK
15	5	2,210 ms	OK
20	6	83,889 ms	OK
25	8	250,305 ms	OK
30	10	395,859 ms	OK
35	11	317,753 ms	OK
40	13	477,693 ms	OK
45	15	207,076 ms	OK
50	15	919,937 ms	OK
55	21	4,208 s	OK
60	20	4,219 s	OK
65	19	15,752 s	OK
70	24	2,953 s	OK
75	23	7,142 s	OK
80	20	1,397 s	OK
85	25	26,898 s	OK
90	27	5,297 s	OK
95	27	4,846 s	OK
100	28	17,800 s	OK

TABLE III
Perbandingan Kualitatif Paradigma Penyelesaian Subset Sum

Paradigma	Waktu	Memori	Perilaku, $N \geq 50$	Perilaku, $T > 10^{12}$	Multi-Solusi
Horowitz-Sahni (1974)	$\mathcal{O}(2^{N/2})$	$\mathcal{O}(2^{N/2})$ (> 1 GB)	Cache thrashing	Independen target	$\mathcal{O}(K \cdot 2^{N/2})$
Pisinger minknab (1997)	Pseudo-P.	$\mathcal{O}(N \cdot T)$	Cepat jika T kecil	Gagal/OOM jika $T > 10^9$	Saksi tunggal
DP Bitset Standar	$\mathcal{O}(N \cdot T)$	$\mathcal{O}(T)$	Cepat jika T kecil	Gagal/OOM	Backtrack penuh
Karya ini (Router L3/L4)	Adaptif (P / pseudo-P / 2^{n-m})	≤ 16 MB tabel + $\mathcal{O}(n)$	0,028 ms–26,9 s (Tabel II)	Jalur L4 independen T	L8: $\mathcal{O}(k^4 \log k + (n-k)^4 \log k)$

VIII. Kesimpulan & Penelitian Masa Depan

Kami telah memaparkan Dumb SSP Solver, arsitektur penyelesai eksak multi-paradigma yang menggabungkan memoization tail-table asimetris terbatas memori (L4, 16,00 MB di dalam cache L3), DP bitset paralel-word (L3), pemangkasan sufiks dan kardinalitas (L5a/L5b), titik batas pembatalan kooperatif (L6), verifikasi independen (L7), dan ekstraktor swap jumlah-nol derajat hingga (L8). Arsitektur ini berhasil menyelesaikan instans acak dengan target triliunan hingga ukuran $N = 100$ pada laptop komoditas dengan konsumsi RAM rendah (≤ 21 MB).

Dari sudut pandang praktis, kemampuan menyelesaikan subset sum berskala triliun dalam hitungan detik dengan jejak memori terkendali berpotensi bermanfaat dalam kriptanalisis (evaluasi kriptosistem berbasis knapsack, kalibrasi asumsi kekerasan kisi), protokol *zero-knowledge proof*, alokasi sumber daya komputasi awan (*bin packing*), serta rekonsiliasi transaksi keuangan.

Agenda penelitian masa depan mencakup:

- Paralelisasi multi-core: menerapkan pencarian DFS terdistribusi pada arsitektur CPU banyak-inti.
- Integrasi reduksi kisi: menggabungkan algoritma LLL/BKZ untuk kasus densitas rendah ($d < 0,9408$).
- Sintesis tail-table Gray-code: membangun tabel secara inkremental untuk mereduksi inisialisasi tabel ke $\mathcal{O}(2^m)$.
- Skalabilitas kombinatorial otomatis: ekspansi faktor 2^Z langsung pada evaluasi *batch* otomatis.

References

- [1] R. M. Karp, “Reducibility among combinatorial problems,” in Complexity of Computer Computations, R. E. Miller, J. W. Thatcher, and J. D. Bohlinger, Eds. Plenum Press, 1972, pp. 85–103.
- [2] D. Pisinger, “A minimal algorithm for the 0-1 knapsack problem,” Operations Research, vol. 45, no. 5, pp. 758–767, 1997.

- [3] K. Bringmann, “A near-linear pseudopolynomial time algorithm for subset sum,” in Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). SIAM, 2017, pp. 1073–1084.
- [4] C. Jin and H. Wu, “A simple near-linear pseudopolynomial time randomized algorithm for subset sum,” in 2nd Symposium on Simplicity in Algorithms (SOSA), ser. OASiCS, vol. 69. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, pp. 17:1–17:6.
- [5] L. Chen, J. Lian, Y. Mao, and G. Zhang, “An improved pseudopolynomial time algorithm for subset sum,” in Proceedings of the 65th IEEE Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2024, pp. 2202–2216.
- [6] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM (JACM)*, vol. 21, no. 2, pp. 277–292, 1974.
- [7] R. Schroeppel and A. Shamir, “A $\mathcal{T} = \mathcal{O}(2^{n/2})$, $\mathcal{S} = \mathcal{O}(2^{n/4})$ algorithm for certain NP-complete problems,” *SIAM Journal on Computing*, vol. 10, no. 3, pp. 456–464, 1981.
- [8] J. C. Lagarias and A. M. Odlyzko, “Solving low-density subset sum problems,” *Journal of the ACM (JACM)*, vol. 32, no. 1, pp. 229–246, 1985.
- [9] M. J. Coster, A. Joux, B. A. LaMacchia, A. M. Odlyzko, C.-P. Schnorr, and J. Stern, “Improved low-density subset sum algorithms,” *Computational Complexity*, vol. 2, no. 2, pp. 111–128, 1992.